Week 6: Retrieval-Augmented Generation (RAG)

Python 200 | Mentor Session

The Knowledge Cutoff Problem

Every LLM has a **training cutoff** — it knows nothing about events after that date.

Models also have no access to **private data**: your company's docs, internal databases, customer records.

When a model doesn't know something, it doesn't always say so. It **hallucinates** — generates a plausible-sounding but invented answer.

> **Discussion:** You're building a customer support chatbot for a company with frequently updated policies. What goes wrong if you just use a plain LLM with no extra data access?

Three Ways to Give an LLM New Knowledge

Strategy	How It Works	Best When
Context injection	Add info directly to the prompt	Small, stable facts
Fine-tuning	Retrain the model on your data	Teaching style/tone; expensive
RAG	Retrieve relevant docs at query time	Large or changing knowledge bases

RAG is the default choice for most production applications.

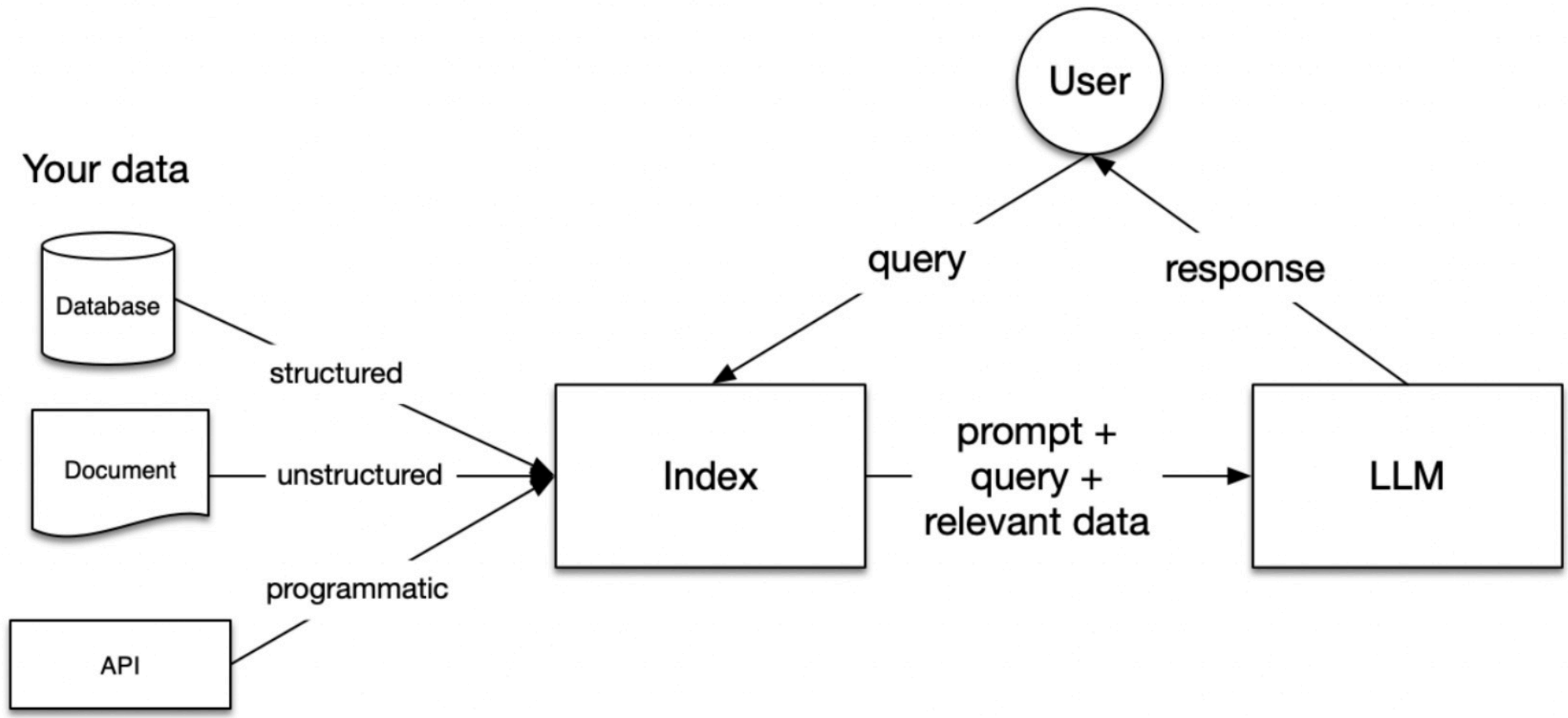
> **Discussion:** Why would you choose RAG over fine-tuning if both can give the model access to new information?

RAG: The Core Pattern

1. **Index** — chunk your documents, store them in a searchable database
2. **Retrieve** — when a user asks a question, find the most relevant chunks
3. **Augment** — inject those chunks into the prompt alongside the question
4. **Generate** — the LLM answers using both its knowledge and the retrieved context

The model's answer is grounded in real documents — not invented.

See next slide for diagram →


User
Your data
query
response
Database
structured
Document
unstructured
Index
prompt +
query +
relevant data
LLM
programmatic
API

Keyword RAG: The Simplest Version

Build a RAG pipeline using plain text search — no embeddings, no database.

Steps:

1. Load documents (PDFs, text files)
2. Extract and chunk the text
3. Search chunks for the words in the query
4. Inject matching chunks into the LLM prompt

This is the "Hello World" of RAG — simple enough to build from scratch in one lesson.

Kaggle Demo: Keyword RAG *(Lesson 2)*

Open: Lesson 2 → "Open in Kaggle" button

Focus on	Skim for now
How documents are chunked and stored	PDF parsing details
The search function and how it matches chunks	File path handling
How retrieved chunks are inserted into the prompt	
Model answer with vs. without context	

Try a query where the answer is in the documents, then one where it isn't.

> **Discussion:** When the model has the right chunk in its context, what does its answer look like compared to when it doesn't?

Where Keywords Fall Short

Keyword search only finds documents that contain your **exact words**.

- Query: `"car insurance"` → misses documents that say `"auto coverage"`
- Query: `"vacation policy"` → misses `"Paid Time Off Guidelines"`
- Query: `"fix the bug"` → misses `"resolve the issue"`

At scale, with varied language, keyword search breaks down.

The solution: search by **meaning**, not by exact words.

Embeddings for Semantic Search

Embed both the query and every document chunk into the same vector space.

- A question about "cars" lands near documents about "automobiles"
- Search by **cosine similarity**: find the vectors closest to the query
- Handles synonyms, paraphrasing, and related concepts naturally

> **Discussion:** We saw embeddings last week as a way to represent meaning as numbers. How does that same idea make search better?

Semantic Search

VS

Keyword Search

lightweight laptop for students

Results include laptops optimized for students, like Chromebooks or budget laptops with long battery life, lightweight designs, and academic tools.

Dell Inspiron 14 - Lightweight, Student Edition

Lenovo Chromebook - Best for Schoolwork

lightweight laptop for students

Results include any laptops with “lightweight” or “student” in the description, potentially showing unrelated products.

Lightweight Gaming Laptop

Student Laptop Bag

pgvector: Storing Vectors in Postgres

pgvector is a Postgres extension that adds vector storage and similarity search.

- Store embeddings as a column in a regular Postgres table
- Run locally via Docker: `docker run -p 5432:5432 pgvector/pgvector`
- Query with `<=>` (cosine distance operator)

Gives you **persistence** — embeddings survive across sessions, unlike an in-memory list.

pgvector: The Query

```
# After storing embeddings, retrieve the 5 most similar chunks
cur.execute("""
    SELECT content FROM documents
    ORDER BY embedding <=> %s
    LIMIT 5
""", (query_embedding,))
```

`<=>` is the cosine distance operator — closer = more similar.

Kaggle Demo: Semantic RAG *(Lesson 3)*

Open: Lesson 3 → "Open in Kaggle" button

Note: This lesson uses Docker for pgvector — it runs locally, not on Kaggle.

Focus on	Skim for now
How embeddings are generated and stored	Docker setup commands
The similarity search query	pgvector SQL syntax details
How results differ from keyword RAG	

> **Discussion:** You embedded 10,000 documents. A user submits a query. How does the system find the relevant ones quickly? What would make this slow at scale?

LlamaIndex: Production RAG

Building RAG from scratch took ~100 lines of code. With LlamaIndex: ~10.

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader

documents = SimpleDirectoryReader("./data").load_data()
index = VectorStoreIndex.from_documents(documents)

query_engine = index.as_query_engine()
response = query_engine.query("What is the vacation policy?")
```

LlamaIndex handles chunking, embedding, indexing, and retrieval — with production-ready defaults.

> **Discussion:** If LlamaIndex does all of this automatically, why did we spend two lessons building RAG from scratch?

Kaggle Demo: LlamaIndex *(Lesson 4)*

Open: Lesson 4 → "Open in Kaggle" button

Focus on	Skim for now
How few lines of code replace the DIY pipeline	LlamaIndex's internal architecture
The query engine interface	Advanced customization options
Response quality compared to earlier lessons	

Run the same query you tested in Lessons 2 and 3. Compare the code and the answer quality.

> **Discussion:** LlamaIndex is an abstraction over everything we built manually. When would you want to go lower-level? When is the abstraction the right call?

Week 6 Wrap-Up

- **Knowledge cutoff / hallucination** — LLMs know only what they were trained on; RAG gets around this
- **Three strategies** — context injection, fine-tuning, RAG — RAG is the right default for large or changing data
- **Keyword RAG** — search by exact words, inject chunks, generate; simple to build, limited in practice
- **Semantic RAG** — search by meaning using embeddings; finds what keyword search misses
- **pgvector** — vector storage in Postgres; SQL + similarity search = persistent semantic retrieval
- **LlamaIndex** — production framework; 10x less code, battle-tested defaults

Takeaway: RAG is one of the most practical patterns in modern data engineering. Any time someone asks "can we make this LLM aware of our data?" — RAG is almost always the answer.

> **Discussion:** Think of a product you use. Could RAG make it better? What data would you give the model access to?